

Idea TFG

Mario Merino Martín

Descripción Breve	1
Estructura Técnica del Proyecto	1
1. Ingesta y Traducción Automática (De Texto a Lógica).....	1
2. Motor de Ejecución y Resolución (El "Árbitro")	2
2.1. Arquitectura Base y Modelado del Dominio	2
2.1.1. Justificación del paradigma arquitectónico.....	2
2.1.2. Modelado del Dominio: Clases Nucleares	3
2.2. Desarrollo de la Prueba de Concepto (PoC) y Abstracción Matemática	3
2.2.1. Simplificación del dominio y validación del pipeline	3
2.2. Traducción del Entorno: Espacios de Observación y Acción.....	4
2.3. Estandarización Multi-Agente: Desacoplamiento con PettingZoo	4
2.3.1. Justificación del desacoplamiento arquitectónico	5
2.3.2. Implementación del Wrapper PettingZooEnv	5
2.3.3. Vectorización y entrenamiento paralelo masivo	5
2.4. Escalabilidad y Resolución de Reglas Complejas	6
2.4.1. Escalabilidad vertical y representación dinámica del entorno	6
2.4.2. Integración de La Pila (The Stack) y Enmascaramiento de Acciones (Action Masking)	6
2.4.3. Verificación Formal Continua: El Patrón Observer frente a los Bucles	6

3. Módulo de Verificación Formal (Bucles e Indeterminismo).....	7
4. Inteligencia Artificial para Deckbuilding (Generador de Mazos)	7
4.1. Adquisición de Datos y Resolución del Problema de Arranque en Frío (Cold Start) 8	
4.1.1. Evaluación de fuentes de datos y pivotaje arquitectónico	8
4.1.2. Conocimiento A Priori y Heurística Humana	8
4.2. Arquitectura del Motor de Mutación (Selección de Cartas).....	9
4.2.1. Evolución del Operador de Selección: De Ruleta a Softmax	9
4.2.2. El Salto al Estado del Arte: UCB1 con Inferencia Bayesiana.....	10
4.2.3. La Ecuación de Decisión Final	10
4.3. Preservación Estructural: Estratificación Blanda (Soft Stratification)	11
4.3.1. El problema de la coherencia estructural y los roles	11
4.3.2. Implementación de la Estratificación Blanda	12
4.3.3. Reducción de varianza y respaldo académico	12
4.4. Gestión del Volumen: Mutación Gaussiana Dinámica	13
4.4.1. El problema de las variables continuas y el óptimo local.....	13
4.4.2. Implementación de la Mutación Gaussiana (Random Walk).....	13
4.4.3. Control de límites (Culling) y convergencia	13
4.5. Función de Aptitud (Fitness), Varianza y Criterios de Parada	14
4.5.1. Evaluación del entorno mediante Aprendizaje por Refuerzo	14
4.5.2. Control de la Varianza Estocástica mediante Lotes de Época (Epoch Batches)	14

4.5.3. Convergencia y Parada Temprana (Early Stopping)	15
5. Simulación Masiva y Agentes Autónomos.....	15
6. Trazabilidad (Logging) y Análisis de Datos.....	16
Base del trabajo:	16
Tipos:	16

Descripción Breve

El proyecto consiste en el desarrollo de un pipeline computacional completo para auditar y simular las mecánicas complejas de Magic: The Gathering. El sistema procesa automáticamente el lenguaje natural de las cartas convirtiéndolo en autómatas finitos estandarizados bajo el formato **JSON-LD**. Esto permite vincular la extracción de datos de la IA con una **Ontología formal (OWL)** que define estrictamente las reglas del juego, garantizando el rigor formal sin perder el rendimiento computacional necesario para enfrentar a agentes lógicos en miles de simulaciones (Montecarlo). Todo el proceso prescinde de interfaz gráfica a favor de un sistema estricto de *logging* textual. Esto permite extraer análisis estadísticos (ventaja de turno, porcentajes de victoria) y aplicar verificación formal para detectar tanto **bucles infinitos** (problema de la parada) como **estados indeterminados** (lagunas o conflictos en la resolución de las reglas del juego).

Estructura Técnica del Proyecto

A continuación, se desglosa el proyecto en sus 6 módulos principales, detallando la base teórica y su implementación práctica:

1. Ingesta y Traducción Automática (De Texto a Lógica)

- **Teoría subyacente:** *Procesamiento de Lenguaje Natural (PLN)* y *Teoría de Lenguajes Formales*. Trata el texto de las reglas de Magic como una gramática estructurada que debe compilarse a un lenguaje máquina (instrucciones).
- **Implementación:** Se utilizará un modelo de IA (LLM) para leer el dataset bruto. El sistema identificará patrones lógicos (costes, triggers, efectos) y generará un archivo de Representación Intermedia (JSON) estandarizado. Para la validación del dominio de conocimiento, se diseñará una ontología en formato OWL (mediante herramientas como Protégé) que actuará como diccionario maestro y estático de solo-lectura para el LLM. La salida estructurada se realizará en **JSON-LD** aplicando polimorfismo basado en acciones. Para asegurar la integridad del motor de ejecución, se implementará una arquitectura *Human-in-the-Loop*: el sistema no modificará la ontología dinámicamente, sino que inyectará metadatos de validación (Flagging). Si la IA detecta mecánicas ajenas

a la ontología, pondrá la carta en cuarentena en un log de revisión para su posterior modelado manual por el investigador.

2. Motor de Ejecución y Resolución (El "Árbitro")

- **Teoría subyacente:** *Teoría de Autómatas, Máquinas de Estado y Estructuras de Datos*. Modelado del entorno como un sistema dinámico.
- **Implementación:** Un motor programado (ej. en Python) que aplica el Patrón Observador (*Observer*). Gestiona el Estado Global (vidas, zonas de memoria) y "La Pila", implementada como una estructura LIFO (Last-In, First-Out). Cuando ocurre un evento en el juego, el motor avisa a los autómatas (cartas), que transicionan de estado e inyectan sus efectos en la Pila ordenadamente.

2.1. Arquitectura Base y Modelado del Dominio

2.1.1. Justificación del paradigma arquitectónico

Magic: The Gathering es un sistema de estados discretos de extrema complejidad. Un enfoque de programación secuencial clásico (mediante *scripts* lineales) resulta inviable para este dominio, ya que la cantidad de transiciones de estado, interrupciones y excepciones a las reglas generaría un código insostenible y propenso a errores.

Durante la fase de diseño, se evaluó la viabilidad de utilizar el paradigma de la Programación Lógica (como Prolog o Datalog). Teóricamente, este enfoque es ideal para modelar la resolución del reglamento axiomático del juego. Sin embargo, fue descartado en favor de la **Programación Orientada a Objetos (OOP)** por dos motivos críticos de ingeniería:

1. **Rendimiento y mutabilidad:** El sistema requiere ejecutar millones de partidas simuladas por segundo para el entrenamiento de la Inteligencia Artificial. La OOP maneja la mutación de estados en memoria de forma mucho más eficiente que los motores lógicos estandarizados.
2. **Compatibilidad del ecosistema:** Las librerías del estado del arte para Aprendizaje por Refuerzo (*Reinforcement Learning*), como *Gymnasium* o *PettingZoo*, exigen arquitecturas basadas en objetos (clases que heredan propiedades de entornos base).

Por tanto, se adoptó un diseño OOP fuertemente tipado, estructurando el motor en módulos independientes que encapsulan la lógica y se comunican mediante interfaces claras.

2.1.2. Modelado del Dominio: Clases Nucleares

Para garantizar la fidelidad a las reglas del juego y la mantenibilidad del código, la arquitectura base se dividió en tres clases fundamentales:

- **Clase Motor (El Árbitro Global):** Actúa como el núcleo central y director del sistema. Su responsabilidad exclusiva es gestionar el ciclo de vida del turno (Fase Inicial, Principal, Combate, etc.) y validar las acciones. Mantiene el estado global del juego de forma estricta: ninguna entidad puede alterar el estado del tablero directamente; deben solicitar una acción al Motor, quien la evalúa y, si es legal, aplica la transición de estado.
- **Clase Jugador:** Funciona como un contenedor seguro que encapsula los recursos de cada agente. Maneja variables dinámicas escalares (como el total de Vidas y la reserva de Maná) y gestiona las estructuras de datos que representan las zonas de memoria, tanto privadas (Mano, Biblioteca) como públicas (Cementerio, Campo de Batalla).
- **Clase Carta:** Constituye la unidad atómica de información del sistema. En lugar de codificar lógicas complejas de manera rígida en su instanciación inicial, se diseñó con atributos modulares base (coste de maná, fuerza, resistencia, supertipos). Esta estructura preparó a la clase para ser poblada dinámicamente mediante el *pipeline* de traducción de lenguaje natural, aprovechando el polimorfismo que permite el formato JSON-LD (donde una carta puede comportarse como criatura y artefacto simultáneamente dependiendo de sus metadatos).

2.2. Desarrollo de la Prueba de Concepto (PoC) y Abstracción Matemática

2.2.1. Simplificación del dominio y validación del pipeline

Implementar de forma nativa e inmediata todas las reglas complejas de *Magic: The Gathering* (como las Acciones Basadas en el Estado o las interrupciones en la Pila) presentaba un riesgo arquitectónico elevado. Si la IA fallaba al aprender, habría sido difícil aislar si el error provenía de la red neuronal o de un fallo lógico en el motor de reglas.

Por este motivo, se tomó la decisión estratégica de desarrollar primero una **Prueba de Concepto (PoC)** o Motor Reducido. El objetivo principal de esta fase no era simular el juego perfecto, sino validar la integridad del *pipeline* de comunicación entre la lógica orientada a objetos (OOP) y **Tianshou**, la librería de Aprendizaje por Refuerzo elegida para el proyecto. Para lograrlo, las reglas se simplificaron a interacciones mecánicas fundamentales: robar cartas, jugar tierras y declarar fases de combate directas. Esto

permitió centrar los esfuerzos de desarrollo en el verdadero reto de esta fase: la traducción numérica del estado del tablero.

2.2. Traducción del Entorno: Espacios de Observación y Acción

Para que un agente de Aprendizaje por Refuerzo pueda operar, es necesario abstraer la interfaz del juego en estructuras tensoriales estandarizadas. Se utilizó la API de la librería **Gymnasium** para definir estrictamente cómo la IA percibe el mundo y cómo interactúa con él:

- **El Espacio de Observación (*Observation Space*):** La red neuronal no procesa texto natural ni imágenes. Para proporcionarle "visión" del tablero, se empleó la clase `Box` de *Gymnasium*. El estado global del juego (cartas en mano, vidas, maná disponible, criaturas en el campo de batalla) se aplanó y codificó en un vector numérico unidimensional (por ejemplo, un *array* de 124 dimensiones de tipo flotante). Cada índice de este vector representa una variable crítica y normalizada del estado del juego en un instante de tiempo t , permitiendo a la red neuronal evaluar la situación de forma matemática.
- **El Espacio de Acción (*Action Space*):** De igual manera, la toma de decisiones se abstrajo utilizando la clase `Discrete` de *Gymnasium*. Se definió un espacio de acción de tamaño fijo, proporcionando a la IA un "menú" numerado de jugadas posibles. El agente de IA simplemente devuelve un número entero escalar (por ejemplo, la acción "14"). El motor de prueba actúa entonces como un intérprete: recibe ese escalar, lo mapea a su equivalente lógico dentro de la OOP (por ejemplo, "Lanzar la segunda carta de la mano") y ejecuta la transición de estado correspondiente.

Esta abstracción matemática garantizó que la IA pudiera entrenarse en un entorno puramente numérico, sentando las bases estables necesarias para posteriormente reintroducir la complejidad completa del reglamento de *Magic*.

2.3. Estandarización Multi-Agente: Desacoplamiento con PettingZoo

2.3.1. Justificación del desacoplamiento arquitectónico

Magic: The Gathering es, por definición, un entorno competitivo de múltiples agentes con información imperfecta (cada jugador oculta las cartas de su mano). Conectar el motor de juego directamente a la red neuronal mediante una interfaz de agente único (*Single-Agent*) habría requerido programar heurísticas complejas para simular al oponente, limitando drásticamente el potencial de aprendizaje de la Inteligencia Artificial.

Para evitar este problema de acoplamiento, se decidió no conectar la lógica del juego directamente al algoritmo de IA. En su lugar, se implementó una capa intermedia estandarizada utilizando la librería **PettingZoo**, que actualmente representa el estándar de la industria para entornos de Aprendizaje por Refuerzo Multi-Agente (MARL).

2.3.2. Implementación del Wrapper *PettingZooEnv*

La clase principal del motor de reglas se encapsuló dentro de un *wrapper* heredado de *PettingZooEnv*. Esta decisión arquitectónica logró un **desacoplamiento total** entre el dominio del juego y el dominio del aprendizaje automático.

Gracias a esta capa de abstracción, la red neuronal no necesita comprender las reglas de *Magic*, ni cómo se estructuran los turnos, ni la semántica de las cartas. Su única vía de comunicación se reduce a interactuar con una interfaz estandarizada: observar un estado vectorial numérico, emitir una acción discreta y recibir una recompensa escalar (positiva por la victoria, negativa por la derrota). El *wrapper* se encarga de gestionar la transición de agentes, pausando la ejecución de una red neuronal y cediendo el turno de decisión al oponente de forma transparente.

2.3.3. Vectorización y entrenamiento paralelo masivo

La adopción de la API de *PettingZoo* aportó una ventaja crítica para la viabilidad del proyecto: la capacidad de vectorización nativa. En los algoritmos de Aprendizaje por Refuerzo, la recolección de experiencia (jugar partidas) suele ser el mayor cuello de botella computacional.

Al estandarizar el entorno, fue posible integrarlo con recolectores vectorizados como *DummyVectorEnv* (proporcionado por librerías como *Tianshou* o *Stable-Baselines3*). Esto permite instanciar y clonar el motor de juego múltiples veces en la memoria RAM, ejecutando decenas de partidas simultáneas en un único subproceso (*thread*) o distribuyéndolas en varios núcleos del procesador. De este modo, la IA puede recolectar lotes masivos de experiencia paralela, acelerando exponencialmente la convergencia del entrenamiento sin que la lógica secuencial del motor de reglas actúe como un factor limitante.

2.4. Escalabilidad y Resolución de Reglas Complejas

2.4.1. Escalabilidad vertical y representación dinámica del entorno

El Motor de Prueba (PoC) fue diseñado estrictamente bajo el principio de escalabilidad vertical. Las decisiones arquitectónicas tomadas aseguraron que el salto hacia la

recreación completa de las reglas no rompiera el proceso de entrenamiento de la Inteligencia Artificial.

El principal desafío radicaba en la transición de un vector de estado fijo a una representación dinámica. A medida que el *pipeline* de Procesamiento de Lenguaje Natural (PLN) traduce nuevas cartas con mecánicas complejas al formato JSON-LD/OWL, el tamaño del array de observación debe crecer para acomodar estas nuevas variables lógicas. Gracias al desacoplamiento logrado en fases anteriores, la arquitectura de la red neuronal es capaz de asimilar esta nueva dimensionalidad automáticamente al instanciarse, sin requerir una reescritura del código subyacente del motor.

2.4.2. Integración de La Pila (The Stack) y Enmascaramiento de Acciones (Action Masking)

El núcleo interactivo de *Magic: The Gathering* reside en "La Pila", una estructura de datos tipo LIFO (*Last-In, First-Out*) que gestiona la resolución secuencial de hechizos y habilidades. Introducir esta mecánica en el entorno de Aprendizaje por Refuerzo planteaba un problema grave: la explosión combinatoria de acciones ilegales. Si se permitía a la red neuronal intentar acciones inválidas (como lanzar un Conjuro durante el turno del oponente o sin el maná suficiente) y se la penalizaba ciegamente, el agente tardaría millones de épocas funcionales solo en deducir el reglamento, retrasando el aprendizaje estratégico.

Para solucionar esto, el motor escalado implementó un sistema de **Enmascaramiento de Acciones (Action Masking)**. En cada transición de prioridad, el motor evalúa el estado del juego y genera un vector binario (máscara). Este vector se superpone al menú de acciones de la IA, multiplicando matemáticamente por cero las probabilidades de seleccionar opciones ilegales en ese instante preciso. De este modo, la red neuronal solo evalúa y explora el subconjunto de acciones legalmente válidas, guiando el aprendizaje de forma eficiente y determinista.

2.4.3. Verificación Formal Continua: El Patrón Observer frente a los Bucles

Finalmente, debido a que el sistema de reglas de *Magic* es Turing completo, existe la posibilidad matemática de alcanzar estados de indeterminismo o bucles infinitos (el Problema de la Parada). Permitir que un agente estocástico genere un bucle recursivo en La Pila congelaría el entorno de entrenamiento.

Para blindar la simulación, se implementó un Módulo de Verificación Formal utilizando el **Patrón de Diseño Observer**. Este módulo actúa como un auditor silencioso que se suscribe a los eventos de las clases Carta y Motor. Si el verificador detecta que una

cadena de disparadores (*triggers*) en La Pila se repite secuencialmente sin intervención de decisiones por parte de los jugadores, interrumpe inmediatamente el ciclo.

Al detectar esta falla estructural, el motor aborta la partida, registra la laguna lógica en los *logs* de trazabilidad para su futura auditoría manual, y aplica una **penalización escalar masiva** (ej. recompensa = -100) al agente responsable. Esta retroalimentación negativa extrema condiciona a la IA para que evite activamente ensamblar combinaciones que "rompan" el entorno, priorizando la victoria a través de mecánicas estables y resolubles.

3. Módulo de Verificación Formal (Bucles e Indeterminismo)

- **Teoría subyacente:** *El Problema de la Parada (Halting Problem), Detección de Ciclos en Grafos y Verificación de Software.*
- **Implementación:** * **Bucles:** El motor cuenta con un monitor de recursividad. Si una cadena de disparadores (*triggers*) en la Pila se repite secuencialmente sin intervención de los jugadores, el sistema levanta un flag de "Bucle Infinito".
 - **Indeterminismo:** Si dos autómatas entran en conflicto lógico y el motor de reglas no tiene una instrucción clara para resolverlo, el sistema lanza una "Excepción Lógica" y congela el estado, reportando una laguna en el reglamento (o en la traducción).

4. Inteligencia Artificial para Deckbuilding (Generador de Mazos)

- **Teoría subyacente:** *Optimización Combinatoria y Algoritmos Heurísticos/Genéticos.* Resolver el problema de seleccionar un subconjunto óptimo de elementos (cartas) bajo ciertas restricciones (colores de maná, curva de coste).
- **Implementación:** Un script que lee la base de datos de cartas ya traducidas a JSON y genera mazos de 100 cartas. Usará heurísticas simples (agrupar cartas que comparten etiquetas de sinergia como "ganar vida" o "hacer daño") para asegurar que los mazos sean funcionalmente lógicos antes de la simulación.

4.1. Adquisición de Datos y Resolución del Problema de Arranque en Frío (Cold Start)

4.1.1. Evaluación de fuentes de datos y pivotaje arquitectónico

El objetivo inicial para dotar al sistema de Inteligencia Artificial (IA) de un conocimiento base sólido fue extraer la información de EDHREC, la plataforma de referencia estadística para la construcción de mazos (*deckbuilding*), y usar las cartas proporcionadas por la base de datos de cartas de mtgjson de las que sacaría las necesitadas y las procesaría para el programa. Sin embargo, durante la fase de investigación, se determinó que EDHREC no dispone de una API pública abierta ni almacena directamente las listas de mazos, sino que agrega datos consultando a plataformas de terceros como Archidekt.

Aunque se exploró la posibilidad de utilizar un *wrapper* no oficial para raspar (*scrape*) los datos agregados de EDHREC, se concluyó que este enfoque presentaba limitaciones estructurales. Se tomó la decisión arquitectónica de desarrollar un *script* propio para realizar llamadas directas a la API de Archidekt. Este cambio de plan resultó ser una ventaja estratégica, ya que permitió recopilar no solo las listas principales de los mazos, sino también sus *maybeboards* (cartas consideradas, pero, pero finalmente descartadas por los jugadores humanos), además por como está estructurado archidekt también se nos presenta dentro del formato json de las cartas el oracle text de cada carta y unas categorías que le pone previamente archidekt al añadirla en su base de datos por lo que no tendremos que buscar una solución para ello también nos proporciona información refinada de más tipos como keywords que nos puede ser útil a la hora de saber que programar y algunas categorías más específicas que nos pueden servir para recompilar más datos estadísticos al final del proyecto.

4.1.2. Conocimiento A Priori y Heurística Humana

En el ámbito del aprendizaje automático y los algoritmos evolutivos, uno de los mayores desafíos es el problema del "Arranque en Frío" (*Cold Start*). Si el motor de mutación de la IA comienza a generar mazos de manera completamente aleatoria, el espacio de búsqueda computacional es inabarcable, y el algoritmo perdería cientos de épocas evaluando combinaciones fundamentalmente subóptimas.

Para solucionar esto, los datos masivos extraídos de Archidekt no se impusieron como reglas absolutas que la IA deba seguir ciegamente, sino que se modelaron como **Conocimiento Previo (Priors Bayesianos)**. Esto dota al algoritmo de una intuición o heurística inicial basada en la sabiduría colectiva de los jugadores humanos.

A partir de los archivos JSON recopilados, se establecieron dos métricas fundamentales para dar a la IA sus "instintos" iniciales:

1. **Frecuencia Relativa (F_i):** Define la probabilidad base de encontrar una carta específica en un arquetipo. Si D_i es el número de mazos que contienen la carta y N es el total de mazos analizados, se define como:

$$F_i = \frac{D_i}{N}$$

2. **Media Heurística ($\mu_{H,i}$):** Aplicada a cartas sin límite de copias (como las tierras básicas), calcula el número medio de copias que los jugadores suelen incluir. Si S_i es la suma total de copias en todos los mazos:

$$\mu_{H,i} = \frac{S_i}{D_i}$$

Con esto, teníamos una estadística descriptiva robusta, pero aún necesitábamos un motor para tomar decisiones.

4.2. Arquitectura del Motor de Mutación (Selección de Cartas)

4.2.1. Evolución del Operador de Selección: De Ruleta a Softmax

El primer enfoque para decidir qué cartas debía introducir la IA en el mazo fue utilizar métodos probabilísticos clásicos. Inicialmente, se evaluó una **Selección por Ruleta**, donde la probabilidad de elegir una carta era directamente proporcional a su uso histórico:

$$P_{selec}(i) = \frac{F_i}{\sum F_j}$$

Sin embargo, este método era ineficiente, ya que tendía a estancarse rápidamente en las cartas más populares, limitando la exploración. Para solucionarlo, se propuso evolucionar hacia una **Función Softmax**. Esto introducía el concepto de *Simulated Annealing*

(Recocido Simulado) mediante un hiperparámetro de temperatura (τ), dando control sobre cuándo explorar y cuándo explotar:

$$P(i) = \frac{e^{F_i/\tau}}{\sum e^{F_j/\tau}}$$

El problema descubierto fue computacional: evaluar miles de cartas mediante vectores exponenciales en simulaciones masivas (Montecarlo) iba a ser un proceso excesivamente costoso y lento para el sistema.

4.2.2. El Salto al Estado del Arte: UCB1 con Inferencia Bayesiana

Para resolver las deficiencias computacionales de Softmax, se abandonó la probabilidad pura en favor de un enfoque determinista bajo incertidumbre: el problema del *Multi-Armed Bandit*. Se decidió implementar el algoritmo **Upper Confidence Bound (UCB1)**.

Para solucionar definitivamente el Arranque en Frío, en lugar de empezar de cero, inyectamos la heurística humana calculada en la Fase 1 (la frecuencia f_i). Esto se logró definiendo la tasa de victorias esperada (μ^i) e introduciendo un hiperparámetro K que actúa como "pseudo-observaciones" (partidas fantasma iniciales). Un valor de K alto indica una gran confianza en el conocimiento a priori humano, requiriendo que la IA juegue un gran volumen de partidas simuladas para poder sobrescribir dicha heurística si encuentra un subconjunto de cartas más eficiente. Por el contrario, conforme aumenta el número real de partidas simuladas (n), el peso relativo de K se diluye, permitiendo que las verdaderas métricas de rendimiento dicten la dirección de la evolución del mazo:

$$\hat{\mu}_i = \frac{w_i + K \cdot f_i}{n_i + K}$$

Donde w_i son las victorias empíricas reales de la IA y n_i las veces que la IA ha probado esa carta. El valor K evita que la IA pruebe combinaciones inútiles al principio, dándole un ancla estadística.

4.2.3. La Ecuación de Decisión Final

Finalmente, el algoritmo UCB1 suma la tasa de victorias esperada a un bono de exploración. Este bono aprovecha la incertidumbre, dictada por el logaritmo de las partidas totales simuladas (T), garantizando que ninguna carta quede sin evaluar a largo plazo:

$$UCB_i = \hat{\mu}_i + c\sqrt{\frac{\ln(T)}{n_i + K}}$$

Con esta ecuación matemática, la IA equilibra perfectamente la **Explotación** (aprovechar cartas con alto *winrate*) y la **Exploración** (forzar la prueba de cartas menos usadas), logrando un rendimiento superior sin el coste computacional prohibitivo de los métodos anteriores.

1. **Explotación:** Maximiza el uso de aquellas cartas que han demostrado empíricamente un alto *winrate* en las simulaciones previas.
2. **Exploración:** Fuerza al sistema a probar cartas menos utilizadas aprovechando la incertidumbre estadística. A medida que una carta es ignorada, su margen de incertidumbre crece, obligando al algoritmo a seleccionarla eventualmente para actualizar su valor real.

4.3. Preservación Estructural: Estratificación Blanda (Soft Stratification)

4.3.1. El problema de la coherencia estructural y los roles

Una vez definido el motor de toma de decisiones mediante UCB1, surgió un problema crítico de dominio: un mazo de *Magic: The Gathering* no es solo una colección de cartas con un alto índice de victorias (*winrate*), sino un ecosistema estructurado donde cada pieza cumple una función específica (aceleración de maná, eliminación de amenazas, robo de cartas, etc.).

Si la Inteligencia Artificial operase con libertad absoluta basándose únicamente en el valor de UCB1, corría el riesgo de colapsar la estructura del mazo. Por ejemplo, podría sustituir un artefacto de coste 1 por una criatura de coste 8 simplemente porque esta última tiene una estadística de victorias abrumadora en otras simulaciones, destruyendo así la curva de maná y la viabilidad del mazo. Sin embargo, imponer una restricción estricta (obligar a

que una carta solo pueda cambiarse por otra exactamente igual) impediría que la IA corrigiera los errores de diseño del mazo humano original.

4.3.2. Implementación de la Estratificación Blanda

Para resolver este dilema, se diseñó un mecanismo de **Estratificación Blanda** (*Soft Stratification*). Este enfoque agrupa el espacio de búsqueda por categorías (tipificadas previamente en la base de datos como artefactos, criaturas, encantamientos, etc.), pero permite la permeabilidad entre ellas bajo ciertas condiciones.

Matemáticamente, se introdujo una penalización/bonificación categórica a la puntuación del algoritmo UCB1, utilizando la función **Delta de Kronecker** (δ) y un hiperparámetro de flexibilidad temporal o sesgo (λ). La puntuación final (S_i) de una carta candidata para sustituir a una carta objetivo se calcula de la siguiente manera:

$$S_i = UCB_i + \lambda \cdot \delta(C_i, C_k)$$

Donde:

- UCB_i es el valor de exploración/explotación de la carta candidata.
- C_i es la categoría o rol de la carta candidata.
- C_k es la categoría de la carta que va a ser reemplazada en el mazo.
- $\delta(C_i, C_k)$ devuelve 1 si las categorías coinciden, y 0 si son diferentes.
- λ es el peso que el sistema le da a mantener la estructura original.

El veredicto final sobre qué carta entra en la "Piscina de Mutación" se resuelve seleccionando aquella que maximiza esta puntuación sesgada:

$$i^* = \operatorname{argmax}(S_i)$$

4.3.3. Reducción de varianza y respaldo académico

El uso de esta técnica está respaldado académicamente por los principios de la *Stratified Bayesian Optimization* (Toscano-Palmerin y Frazier, 2016). Como demuestran los autores, agrupar el espacio de búsqueda por atributos categóricos en funciones de evaluación

costosas —como las evaluaciones de recompensas episódicas generadas por los agentes de *Reinforcement Learning* en el entorno de simulación de este proyecto— reduce significativamente la varianza estocástica y acelera la convergencia hacia la política óptima, justificando así la inclusión del sesgo categórico en el *pipeline* evolutivo.

4.4. Gestión del Volumen: Mutación Gaussiana Dinámica

4.4.1. El problema de las variables continuas y el óptimo local

La arquitectura de selección basada en UCB1 resulta altamente eficaz para evaluar la viabilidad de cartas bajo la restricción *Singleton* (decisiones binarias de inclusión o exclusión). Sin embargo, el dominio de *Magic: The Gathering* presenta un reto adicional: la gestión de cartas sin límite de copias. El caso más evidente son las Tierras Básicas, cuya cantidad (el volumen de la base de maná) es un factor determinante en la consistencia matemática del mazo, así como cartas excepcionales que alteran las reglas de construcción (ej. *Relentless Rats*).

Tratar de optimizar la cantidad de estas cartas mediante los operadores binarios anteriores provocaría un estancamiento en óptimos locales, ya que el algoritmo no tendría un mecanismo fluido para escalar o reducir el número de copias de forma secuencial y orgánica.

4.4.2. Implementación de la Mutación Gaussiana (*Random Walk*)

Para resolver este problema, se diseñó un operador de mutación específico para variables de cantidad, basado en una aproximación de **Distribución Normal o Campana de Gauss**. En lugar de asignar valores arbitrarios, el sistema realiza un *Random Walk* (camino aleatorio) alrededor del estado actual del individuo en cada generación.

El concepto central es que la cantidad inicial para la "Generación 0" viene dada por el Promedio Heurístico Humano ($\mu_{H,i}$), calculado en la fase de extracción de datos. A partir de ahí, en generaciones posteriores, la mutación de la cantidad se calcula utilizando una distribución normal centrada en la cantidad actual que posee el mazo.

Matemáticamente, si x_i es el número de copias actual de la carta i , la nueva cantidad propuesta x'_i se define como:

$$x'_i = \max(0, \text{round}(\mathcal{N}(\mu = x_i, \sigma^2)))$$

4.4.3. Control de límites (*Culling*) y convergencia

En la fórmula anterior, la varianza (σ^2) actúa como el hiperparámetro que dicta el tamaño del paso evolutivo. Para garantizar la coherencia del entorno, se aplicó una función de redondeo (round) para mantener el dominio en números enteros (no se pueden tener fracciones de cartas), y una función de límite inferior (*Culling* mediante $\max(0, \dots)$) para asegurar que el sistema jamás intente instanciar una cantidad negativa de copias.

Este mecanismo iterativo proporciona una enorme ventaja adaptativa. Permite a la IA probar variaciones incrementales sutiles (± 1 , ± 2 tierras). Posteriormente, mediante las recompensas generadas por el agente de *Reinforcement Learning* durante las partidas, la selección natural empujará la cantidad de copias hacia el óptimo global necesario para que el mazo funcione, o la reducirá orgánicamente a 0 si la carta resulta ser ineficiente para la estrategia descubierta por el modelo.

4.5. Función de Aptitud (Fitness), Varianza y Criterios de Parada

4.5.1. Evaluación del entorno mediante Aprendizaje por Refuerzo

Para que el motor de mutación (UCB1) pueda actualizar sus métricas y decidir qué cartas son óptimas, necesita una señal de recompensa clara. En lugar de utilizar simulaciones estocásticas de fuerza bruta, el sistema de evaluación se integró con un entorno de **Aprendizaje por Refuerzo (*Reinforcement Learning* - RL).**

En este paradigma, el mazo generado actúa como el espacio de estados y acciones disponible para un agente autónomo. El agente interactúa con el motor de reglas (el "Árbitro"), ejecutando una política de decisiones (π) para maximizar una señal de recompensa acumulativa, que en este contexto se traduce en la victoria de la partida. La **Función de Aptitud (*Fitness Function*)** de un mazo no es simplemente un valor estático, sino el rendimiento empírico que el agente de RL logra alcanzar al pilotarlo contra un oponente de referencia (*baseline*) o en un entorno de auto-juego (*Self-Play*).

4.5.2. Control de la Varianza Estocástica mediante Lotes de Época (*Epoch Batches*)

El entorno de *Magic: The Gathering* es inherentemente estocástico debido a la varianza en el robo de cartas y la aleatoriedad inicial de la baraja. Esta varianza presenta un riesgo crítico para el algoritmo de mutación: una mala racha de robos puede engañar a la IA, haciéndole creer que un mazo excepcionalmente bien construido es deficiente simplemente por haber perdido una partida aislada.

Para aislar la calidad estructural del mazo del "ruido" estocástico, la evaluación de la Función de Aptitud se realiza mediante **Lotes de Época (Epoch Batches)**. Cada mazo mutado no se evalúa en una única partida, sino que el agente de RL disputa un lote fijo de N episodios (por ejemplo, 100 partidas). La aptitud real del mazo $F(d)$ se calcula como la esperanza matemática de sus victorias, aproximada mediante el promedio del *winrate* en el lote:

$$F(d) = \frac{1}{N} \sum_{j=1}^N W_j$$

Donde W_j es el resultado binario (1 para victoria, 0 para derrota) del episodio j . Es este valor $F(d)$ el que retroalimenta la ecuación UCB1 descrita en la sección 4.2, actualizando la tasa de victorias esperada (μ^i) de las cartas involucradas.

4.5.3. Convergencia y Parada Temprana (Early Stopping)

Dado que el espacio de búsqueda combinatorio de *Magic* es astronómico, ejecutar el algoritmo evolutivo indefinidamente resultaría ineficiente. Por tanto, se implementó un criterio de **Parada Temprana (Early Stopping)** basado en el análisis de la curva de aprendizaje.

El algoritmo monitoriza el *winrate* máximo histórico de la población de mazos a lo largo de las generaciones. Se considera que el sistema ha alcanzado la convergencia ("optimización heurística") cuando la curva de la Función de Aptitud entra en una fase asintótica. Matemáticamente, si la mejora del *winrate* máximo (ΔF_{max}) durante un número definido de generaciones consecutivas (g) es inferior a un umbral de tolerancia (ϵ), el entrenamiento se detiene:

$$\Delta F_{max} < \epsilon \quad \text{durante } g \text{ generaciones}$$

Este estancamiento temporal indica que el agente de RL ha encontrado un óptimo (ya sea local o global) dadas las restricciones de la base de datos, dando por finalizado el proceso de *deckbuilding* de esa iteración.

5. Simulación Masiva y Agentes Autónomos

- **Teoría subyacente:** Simulación de sistemas estocásticos (robo de cartas) bajo condiciones de lógica determinista.
- **Implementación:** Ejecución *headless* (sin interfaz gráfica) donde dos agentes (bots) juegan entre sí. Los agentes seguirán un algoritmo de reinforcement learning para tomar decisiones legales validadas por el Motor. Se simularán miles de partidas en segundos.

6. Trazabilidad (Logging) y Análisis de Datos

- **Teoría subyacente:** *Event Sourcing, Auditoría de Sistemas y Estadística Computacional.*
- **Implementación:** No habrá interfaz visual. Cada partida generará un archivo de log secuencial (ej. .txt o .jsonl) que registrará cada mínima transición de estado (ej: "Turno 2 - Jugador 1 lanza Hechizo X -> Pila -> Resuelve -> Vida J2 baja a 17"). Posteriormente, una librería de análisis de datos leerá estos logs para extraer métricas: Win Rate de los mazos, ventaja probabilística de jugar primero (Play/Draw) y reporte de las cartas que más fallos/bucles generan.

Memoria

MEMORIA 1

Cambios/correcciones que hicimos en el motor/entorno (PoC)

1) Observación numérica + máscara de acciones (action masking “real”)

Qué: el entorno MagicEnv ahora devuelve observaciones como un diccionario con:

observation: tu vector numérico (124 floats)

action_mask: un vector binario de tamaño 50 que marca qué acciones son legales en ese estado

Por qué: en juegos con muchas acciones ilegales, entrenar sin máscara hace que el agente pierda muchísimo tiempo probando acciones inválidas y el aprendizaje se vuelve ineficiente.

Dónde: motor_prueba/env_magic.py (método observe() y observation_space tipo Dict).

2) Métrica de acciones inválidas

Qué: contamos cuántas veces cada agente intenta una acción inválida y lo registramos en info y en logs.

Por qué: te permite medir cuantitativamente el impacto del action masking (en memoria queda muy bien como métrica “de ingeniería”).

Dónde: motor_prueba/env_magic.py (invalid_action_count, invalid_action).

3) Recompensa densa vs escasa (configurable)

Qué: añadimos un modo reward_mode:

dense: pequeñas recompensas por jugar cartas/atacar/bloquear/daño

sparse: (casi) solo victoria/derrota

Por qué: para poder experimentar y justificar por qué en PoC suele convenir recompensa densa (mejor señal de aprendizaje).

Dónde: motor_prueba/motor.py (uso de reward_mode).

4) Silenciar prints y trazabilidad

Qué: añadimos `verbose=False` para que el entrenamiento no se ahogue en salida de consola, y un logger JSONL para registrar eventos de forma estructurada.

Por qué: mejora el rendimiento del entrenamiento y te da datos reproducibles/analizables.

Dónde:

`motor_prueba/motor.py`, `motor_prueba/jugador.py`, `motor_prueba/env_magic.py`

`motor_prueba/logging_utils.py` (logger JSONL eficiente)

5) Winrate y longitud de episodios (para gráficas)

Qué: al terminar una partida, el entorno registra un evento `episode_end` con:

ganador (`winner`), `win0` (si gana `player_0`), `episode_len_steps`

Por qué: sin esto, el “winrate” y la longitud media de episodios no se pueden graficar fácilmente.

Dónde: `motor_prueba/env_magic.py` (log `episode_end`).

Cambios/correcciones en el entrenamiento (Tianshou)

6) Adaptación de la red a observaciones multi-agente (arreglo del `str/ndim`)

Qué: añadimos un wrapper `ObsExtractorNet` para que, si la observación llega como objeto estructurado, la red use solo la parte numérica.

Por qué: evita que el modelo intente convertir `agent_id` (string) a tensor.

Dónde: `motor_prueba/train.py` (`ObsExtractorNet`).

7) Policy que respeta la máscara

Qué: `MaskedProbabilisticActorPolicy` aplica `batch.obs.mask` para que las acciones ilegales tengan probabilidad 0.

Por qué: el action masking del entorno debe llegar hasta la decisión final del actor para ser efectivo.

Dónde: motor_prueba/train.py.

8) Parametrización por CLI + baseline reproducible

Qué: train.py ahora acepta flags como --run-name, --max-epochs, --epoch-steps, --disable-mask, --reward-mode, etc.

Por qué: para correr experimentos controlados y repetibles (esto también es muy “TFG friendly”).

Dónde: motor_prueba/train.py.

9) TensorBoard logging (curvas automáticas)

Qué: se añadió TensorboardLogger de Tianshou para que el entrenamiento escriba scalars en logs/tb/<run_name>.

Por qué: permite visualizar reward/test y otras métricas en el UI de TensorBoard.

Dónde: motor_prueba/train.py (logger en OnPolicyTrainerParams).

10) Marcadores de epoch en JSONL

Qué: añadimos eventos epoch_start y epoch_end en el JSONL mediante callbacks (training_fn, test_fn).

Por qué: para poder agrupar métricas derivadas del JSONL (p.ej. invalid_ratio) por epoch.

Dónde: motor_prueba/train.py.

Integración mínima con datos reales (Archidekt)

11) Cargar mazos reales “compatibles PoC”

Qué: construimos un loader que toma tu dataset de Archidekt y genera decks PoC usando solo Lands y Creatures (lo demás se ignora por ahora).

Por qué: demuestra escalabilidad hacia datos reales sin exigir soportar todas las reglas/carta-text todavía.

Dónde:

motor_prueba/deck_loader.py

motor_prueba/env_magic.py (usa dataset_path)

Visualización (lo que me pediste para entender y “ver” el aprendizaje)

12) Graficador seleccionable (curvas)

Qué: visualize_training.py genera PNG con:

reward (desde TensorBoard)

invalid_ratio (desde JSONL)

episode_length y winrate (desde episode_end en JSONL)

Dónde: motor_prueba/visualize_training.py

13) Grafo real de la red (diagrama)

Qué: visualize_network.py intenta generar un PNG del grafo del actor y/o critic usando torchviz (y Graphviz del sistema). Si no están, hace fallback textual / .gv.

Dónde: motor_prueba/visualize_network.py

Doc: motor_prueba/README_viz.md

MEMORIA 2

Implementación y herramientas desarrolladas o ampliadas

1. Entrenamiento (train.py)

PPO + Tianshou 2.x con entorno PettingZoo (PettingZooEnv + MagicEnv).

Action masking opcional (política que anula acciones ilegales y renormaliza la distribución).

Logging duplicado:

JSONL por run (logs/<run>.jsonl) con eventos de pasos, turnos y, tras ampliar el entorno, episode_end con longitud de episodio, ganador, win0, truncado, vidas, etc.

TensorBoard en logs/tb/<run>/ con creación explícita de carpetas y cierre/flush del SummaryWriter para asegurar archivos de eventos.

--max-episode-steps: truncación de episodios largos para que existan finales de episodio registrables (necesario para winrate / longitud cuando el juego no “mata” rápido).

Valores por defecto de entrenamiento más largos (más epochs, pasos por epoch, más episodios de test) para curvas más estables.

Problema detectado y corregido (métrica, no el aprendizaje del modelo): con reward sparse y fin de partida tipo +10 / -10, reducir los retornos multi-agente con la media entre jugadores hace que la estadística de test sea ~0 aunque haya ganador. Se documentó y se cambió la reducción usada solo para estadísticas/logs en sparse a algo interpretable (p. ej. media de |retorno| por episodio), sin pretender que el “0” anterior fuera que “no aprendía”.

Ruido en consola: mensajes repetidos de Tianshou sobre secuencias 2D; se filtraron (warnings / líneas impresas) para no ocultar el progreso del entrenamiento.

2. Entorno (env_magic.py + motor.py)

motor.py: reward densa (premios por jugar carta, atacar, bloquear, daño a vida) vs sparse (casi solo señal al final vía bonus en el entorno al colapsar vida).

env_magic.py: registro de episode_end; con truncación por pasos máximos, finales truncados con interpretación de resultado por diferencia de vidas o empate, para poder medir métricas por episodio aunque no haya muerte real.

3. Visualización de entrenamiento (visualize_training.py)

Lectura de métricas desde TensorBoard o JSONL (según existan datos y la fuente elegida).

Métricas: reward, invalid_ratio, episode_length, winrate (este último anclado a win0, victorias del asiento player_0).

Modo interactivo (elegir log, métrica, eje temporal, salida, suavizado) y modo CLI completo; argumento tipo run posicional.

--reward-source: evitar confusiones entre curva “limpia” de test (TensorBoard) y reward por paso del JSONL (muy ruidosa).

Suavizado (--smooth) para rewards escalonadas / ruidosas.

Corrección de bug: un return prematuro impedía guardar el PNG en ciertos modos de reward por epoch.

4. Visualización de redes (visualize_network.py)

Exportación del grafo actor (50 acciones, capas 128) y critic (salida escalar) con torchviz + Graphviz; explicación pedagógica de softmax en el actor como parte de la política categórica bajo PPO, no como “sustituto” de PPO.

5. Batalla entre checkpoints (match_checkpoints.py)

Enfrentamiento sin entrenar entre dos .pth (--p0-ckpt, --p1-ckpt), player_0 vs player_1 con distinta política.

Bucle sobre MagicEnv nativo (se descartó el wrapper de Tianshou en el loop de juego por problemas de API con agentes terminados / step(None)).

Opción --swap para alternar qué checkpoint juega como P0 y mitigar sesgo de turno.

Resumen de victorias y empates/truncados.

6. Otros

analyze_logs.py: análisis rápido de JSONL existente (patrón de selector de ficheros reutilizado mentalmente en el modo interactivo de visualize_training).

Decisiones tomadas (y justificación breve para memoria)

Truncación de episodio

Priorizar métricas por episodio (winrate, longitud) frente a partidas que nunca terminan; aceptar que episodios truncados no equivalen a “victoria limpia”.

winrate en self-play

Medir victorias de player_0; en self-play simétrico el valor oscila cerca del 50% salvo sesgos — útil para detectar asimetrías, menos para afirmar “mejora absoluta” sin baseline externo.

Dos fuentes de reward en gráficos

TensorBoard (test) vs JSONL (step) miden cosas distintas; se hizo explícita la elección de fuente para no comparar gráficos incomparables.

Dense vs sparse como eje experimental

Dense como curriculum / shaping para acciones básicas; riesgo observado: optimizar micro-recompensas (trades, duración) sin alinear con ganar. Sparse fuerza alineación con resultado terminal, a costa de señal más escasa y aprendizaje potencialmente más lento.

Evaluación head-to-head

Complementar self-play con duelo entre dos políticas guardadas (dense vs sparse): respuesta directa a “¿cuál juega mejor en este motor?”.

Hallazgos / lecciones aprendidas (propias de discusión)

Plateau de longitud de episodio cerca del máximo con reward densa y self-play: compatible con estancamiento por trades y shaping, no necesariamente con “no aprende”.

Recompensa densa puede desalinearse del objetivo de victoria (incentivos por atacar/bloquear/jugar que se pagan mutuamente en combates simétricos).

test_reward ≈ 0 en sparse con reducción mean(agentes) no implica episodios sin reward terminal: es artefacto de agregación en juego suma-cero a dos jugadores.

Resultados del duelo (p. ej. sparse ganando claramente a dense en N partidas sin truncados, con swap): evidencia empírica concreta de que, en tu setup y con tus checkpoints, la política sparse resultó más fuerte en el enfrentamiento directo — conviene matizar con seeds, duración de entreno e igualdad de condiciones (misma máscara, mismos límites de pasos).

--- Resumen --- modelo_magic_dense_1 victorias: 14 modelo_magic_sparse_1 victorias: 36 Empates/truncados sin ganador: 0 / 50 Win rate modelo_magic_dense_1 (sobre partidas con ganador): 0.280 como P0: modelo_magic_dense_1 9W, modelo_magic_sparse_1 20W | como P1: modelo_magic_dense_1 5W, modelo_magic_sparse_1 16W